

Comparatif Avant / Après : Intégration de l'API Fnac Darty

Ce document met en évidence l'évolution globale du code entre le projet initial et l'intégration complète de la fonctionnalité d'export vers la marketplace Fnac Darty (simulation).

1. Base de Données & Modèle (Produit)

Avant

La table `produits` ne contenait que les informations de base pour la vente directe via votre Drive (libellé, prix, stock, image...). Il n'y avait aucun moyen de distinguer un produit périssable d'un produit non périssable.

Après

1. Migration : Création d'une nouvelle migration pour ajouter les champs spécifiques au fonctionnement de la Marketplace.

```
// database/migrations/..._add_fnacdarty_fields_to_produits_table.php
$table->boolean('is_non_perissable')->default(false);
$table->boolean('export_fnacdarty')->default(false);
$table->string('fnacdarty_category')->nullable();
$table->string('export_status')->nullable();
```

2. Modèle : Mise à jour du modèle [Produit](file:///c:/code/Epreuve_E6_Legere/app/Models/Produit.php#46-103) pour autoriser l'assignation de masse (`$fillable`) de ces nouveaux champs et caster les booléens correctement.

```
// app/Models/Produit.php
protected $fillable = [
    // ... anciens champs ...
    'is_non_perissable', 'export_fnacdarty', 'fnacdarty_category', 'export_status'
];

protected $casts = [
    'is_non_perissable' => 'boolean',
    'export_fnacdarty' => 'boolean',
];
```

2. Logique d'Export et Communication API (Nouveau)

Avant

L'application était complètement isolée. Aucune logique de communication externe avec une autre plateforme (API) n'existait.

Après

1. Service d'Export : Création du [FnacDartyService]

(file:///c:/code/Epreuve_E6_Legere/app/Services/FnacDartyService.php#9-64) pour transformer un produit de la BDD vers le format exigé par l'API Fnac Darty (Mapping des champs).

```
// app/Services/FnacDartyService.php
$payload = [
    'sku' => $produit->reference,
    'product_title' => $produit->libelle,
    'price' => $produit->prix_ht, // Mapping tarif HT
    'quantity' => $produit->stock,
    'descr' => $produit->description,
    'category' => $produit->fnacdarty_category
];
// Envoi via client HTTP natif de Laravel
$response = Http::post('http://localhost/api/fnacdarty/mock', $payload);
```

2. Mock de l'API (Simulation) : Création d'un contrôleur API factice chargé de recevoir ces requêtes pour simuler la vraie Fnac (sans dépendance réseau).

```
// app/Http/Controllers/Api/FnacDartyMockController.php
public function store(Request $request) {
    // Validation et retour de succès (statut HTTP 200)
    return response()->json([
        'status' => 'success',
        'message' => 'Produit importé dans Fnac Darty'
    ]);
}
```

3. Automatisation de l'Export (Nouveau)

Avant

L'application ne contenait aucune tâche planifiée (CRON) pour effectuer des traitements en arrière-plan sans action humaine.

Après

1. Commande Artisan : Création d'une commande système récupérant tous les produits éligibles pour les exporter à la chaîne sans bloquer la navigation Web.

```
// app/Console/Commands/ExportFnacDartyCommand.php
protected $signature = 'fnacdarty:export';

public function handle(FnacDartyService $exportService) {
    $produitsToExport = Produit::where('is_non_perissable', true)
        ->where('export_fnacdarty', true)->get();

    foreach ($produitsToExport as $produit) {
        // Appel du service et MAJ du statut d'export en BDD
        $exportService->export($produit);
    }
}
```

2. Planification : Programmation de cette commande au niveau du serveur pour une exécution quotidienne hors des pics d'affluence.

```
// routes/console.php
Schedule::command('fnacdarty:export')->dailyAt('02:00');
```

4. Affichage et Parcours Client

Avant

Tous les produits (frais, épicerie, surgelés...) étaient mélangés sans regroupement physique spécifique pour les articles de type "Marketplace / Longue conservation".

Après

1. Route et Contrôleur : Création d'un point d'accès pour exposer uniquement les articles non périssables.

```
// app/Http/Controllers/ProduitsController.php
public function nonPerissables() {
    $produits = Produit::where('is_non_perissable', true)
        ->where('actif', true)->paginate(12);

    return view('produits.non_perissables', compact('produits'));
}
```

2. Intégration Interface Client : Une page dédiée a été construite et liée à la barre de navigation principale (Header) du e-commerce pour que les clients accèdent à ce "Coin Non-Périssable".

```
<!-- resources/views/layouts/navigation.blade.php -->
<a href="{{ route('produits.non-perissables') }}" class="text-green-300 font-
bold">Non Périssables</a>
```

Résultat : L'application a maintenant un système complet d'import/export de données asynchrone pour gérer son catalogue vers l'extérieur, tout en offrant une nouvelle vitrine structurée à ses propres clients.